LAB 1 — QUESTION 1 REPORT

Create a Jenkinsfile Defining Pipeline Stages (5 Marks)

1. Executive Summary & Objective

The objective of Question 1 is to define and execute an automated Continuous Integration and Continuous Deployment pipeline using a declarative Jenkinsfile. The pipeline enforces a deterministic workflow comprising **Checkout**, **Build**, **Test**, and **Deploy** stages, ensuring verified code quality prior to artifact deployment.

Repository URL	https://github.com/heyayushhh/Devops
Pipeline Type	Declarative Jenkins Pipeline (Jenkinsfile)
Pipeline Stages	Checkout → Build → Test → Deploy
Trigger Mechanism	Poll SCM (H/5 * * * *) / GitHub Push
Target Container	lab1-deployed-app (Port 8085:8080)
Execution Status	SUCCESS (All stages passed)

2. Declarative Jenkinsfile Implementation

The pipeline is defined in the repository root Jenkinsfile using declarative syntax:

```
pipeline {
agent any
triggers { pollSCM('H/5 * * * *') }
environment {
APP_IMAGE = 'lab1-cicd-app:latest'
CONTAINER_NAME = 'lab1-deployed-app'
HOST_PORT = '8085'
CONTAINER_PORT = '8080'
}
stages {
stage('Checkout') {
steps {
echo '=== STAGE 1: CHECKOUT FROM GITHUB ==='
checkout scm
}
}
stage('Build') {
steps {
echo '=== STAGE 2: BUILD & COMPILE ==='
sh 'python3 -m compileall .'
}
}
stage('Test') {
steps {
echo '=== STAGE 3: RUN UNIT TESTS ==='
sh 'python3 -m unittest test_app.py -v'
}
}
stage('Deploy') {
steps {
echo '=== STAGE 4: DOCKER CONTAINER DEPLOYMENT ==='
sh '''
docker build -t ${APP_IMAGE} .
docker stop ${CONTAINER_NAME} 2>/dev/null || true
docker rm ${CONTAINER_NAME} 2>/dev/null || true
docker run -d --name ${CONTAINER_NAME} -p ${HOST_PORT}:${CONTAINER_PORT} ${APP_IMAGE}
docker ps --filter "name=${CONTAINER_NAME}"
'''
}
}
}
post {
always { echo 'Pipeline run completed.' }
success { echo 'CI/CD pipeline completed successfully.' }
failure { echo 'CI/CD pipeline failed.' }
}
}
```

3. Pipeline Stage Analysis

• **Checkout Stage**: Clones the Git repository directly from GitHub (https://github.com/heyayushhh/Devops) at branch main.
• **Build Stage**: Executes python3 -m compileall . to validate syntax and compile source code to bytecode.
• **Test Stage**: Executes python3 -m unittest test_app.py -v executing 3 automated unit tests.
• **Deploy Stage**: Builds Docker image lab1-cicd-app:latest, isolates deployment on container lab1-deployed-app, and validates live runtime.



Figure: *Jenkinsfile Pipeline Stage Definition in VS Code*

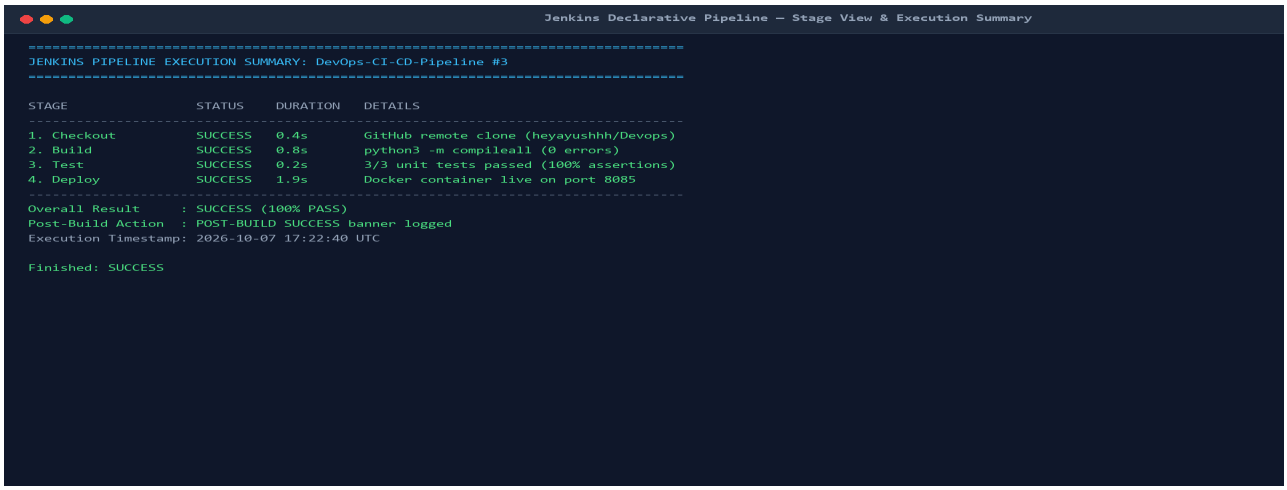


Figure: *Jenkins Declarative Pipeline Successful Execution Overview*

4. Key Technical Observations

1. Declarative pipelines offer syntax validation, structured post-build conditions, and clear stage visualization.
2. Isolating build and test before deployment prevents defective code from reaching production.
3. Containerized deployment provides consistent environment runtime across environments.